

B.TECH. SEM IV

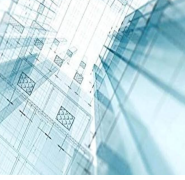
Course- Software Architecture Analysis

Course Code- 24CS02HT0401

Category- Honors



By-
Vaishali Katkar



Software Architecture Analysis

The design phase of software development deals with transforming the customer requirements as described in the SRS documents into a form implementable using a programming language.

The software design process can be divided into the following three levels of design:

1. Interface Design
2. Architectural Design
3. Detailed Design



Elements of a System



1. **Architecture:**

This is the conceptual model that defines the structure, behavior and views of a system.

2. **Modules:**

These are components that handle one specific task in a system. A combination of the modules makes up the system.

3. **Components:**

This provides a particular function or group of related functions. They are made up of modules.

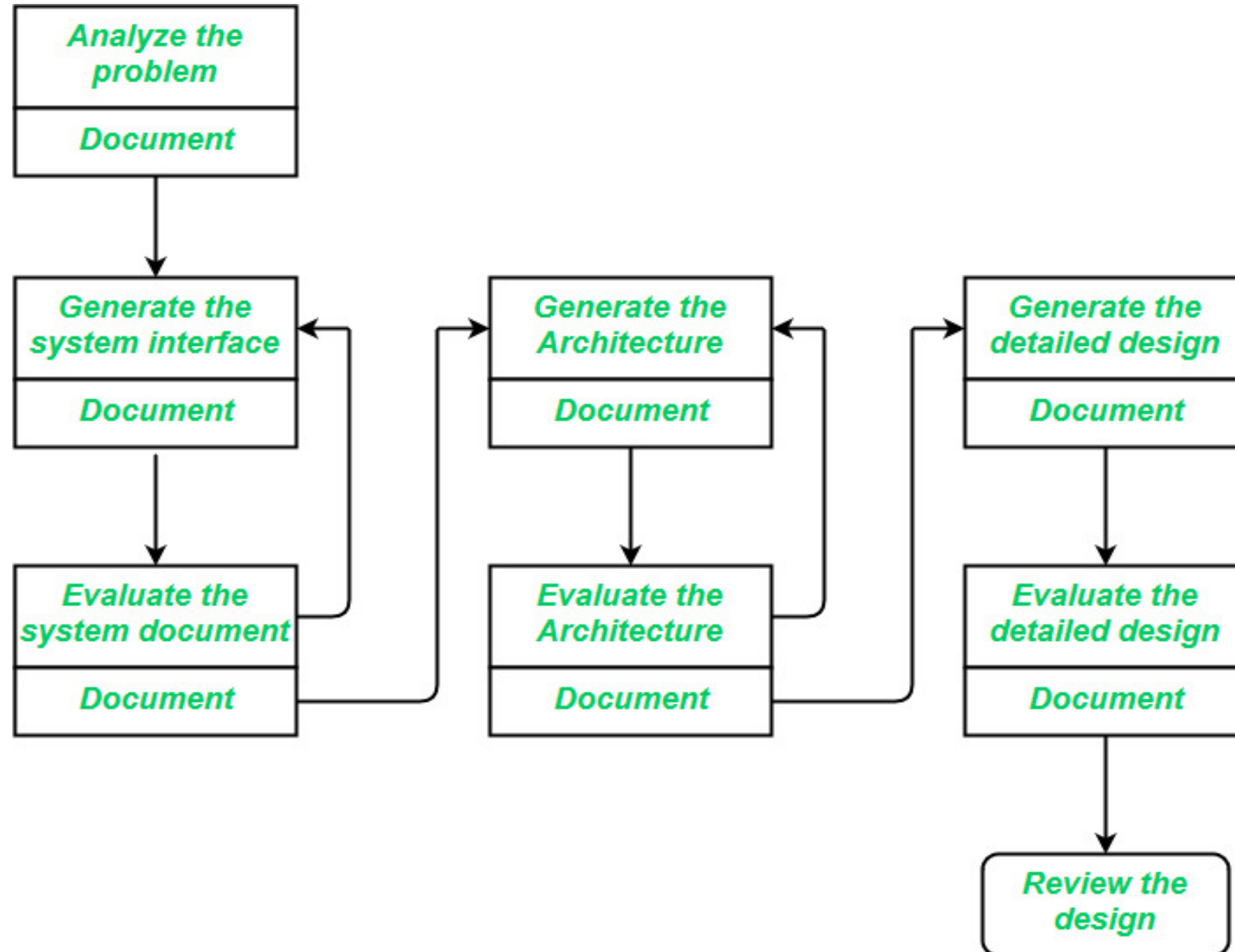
4. **Interfaces:**

This is the shared boundary across which the components of a system exchange information and relate.

5. **Data:**

This is the management of the information and data flow.

Software Design Process



- Interface design is the specification of the interaction between a system and its environment. i.e, during interface design, the internal of the systems are completely ignored, and the system is treated as a **black box**.
- Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts. The design problem statement produced during the problem analysis step should identify the people, other systems, and devices which are collectively called **Agents**.
- Interface design should include the following details:
 - Precise description of events in the environment, or messages from agents to which the system must respond.
 - Precise description of the events or messages that the system must produce.
 - Specification of the data, and the formats of the data coming into and going out of the system.
 - Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

- Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored.
- Issues in architectural design includes:
 - Gross decomposition of the systems into major components.
 - Allocation of functional responsibilities to components.
 - Component Interfaces.
 - Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.
 - Communication and interaction between components.
 - The architectural design adds important details ignored during the interface design. Design of the internals of the major components is ignored until the last phase of the design.

- Detailed design is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures.
- The detailed design may include:
 - Decomposition of major system components into program units.
 - Allocation of functional responsibilities to units.
 - User interfaces.
 - Unit states and state changes.
 - Data and control interaction between units.
 - Data packaging and implementation, including issues of scope and visibility of program elements.
 - Algorithms and data structures.

Software Design Phases



- Software Design:
Creative activity, in which you:
 - Identify software components and their relationships based on requirements.
- Implementation is the process of realizing the design as a program.
- Design may be
 - Documented in UML (or other) models
 - Informal sketches (whiteboard, paper)
 - In the programmer's head.
 - How detailed and formal it is, depends on the process that is in use.

- Functional Decomposition
 - Top down design
- Relational Database Design
- Object-oriented design and UML
 - class diagrams
 - state diagrams
- User Interface design



- Used in structural programming
 - Start with a “main module”
 - Repeatedly decompose into sub-modules.
 - Lowest level modules can be implemented as functions.
- Can be used in Object-oriented design
 - to do initial decomposition of a system
 - to decompose methods that are particularly hard to implement.

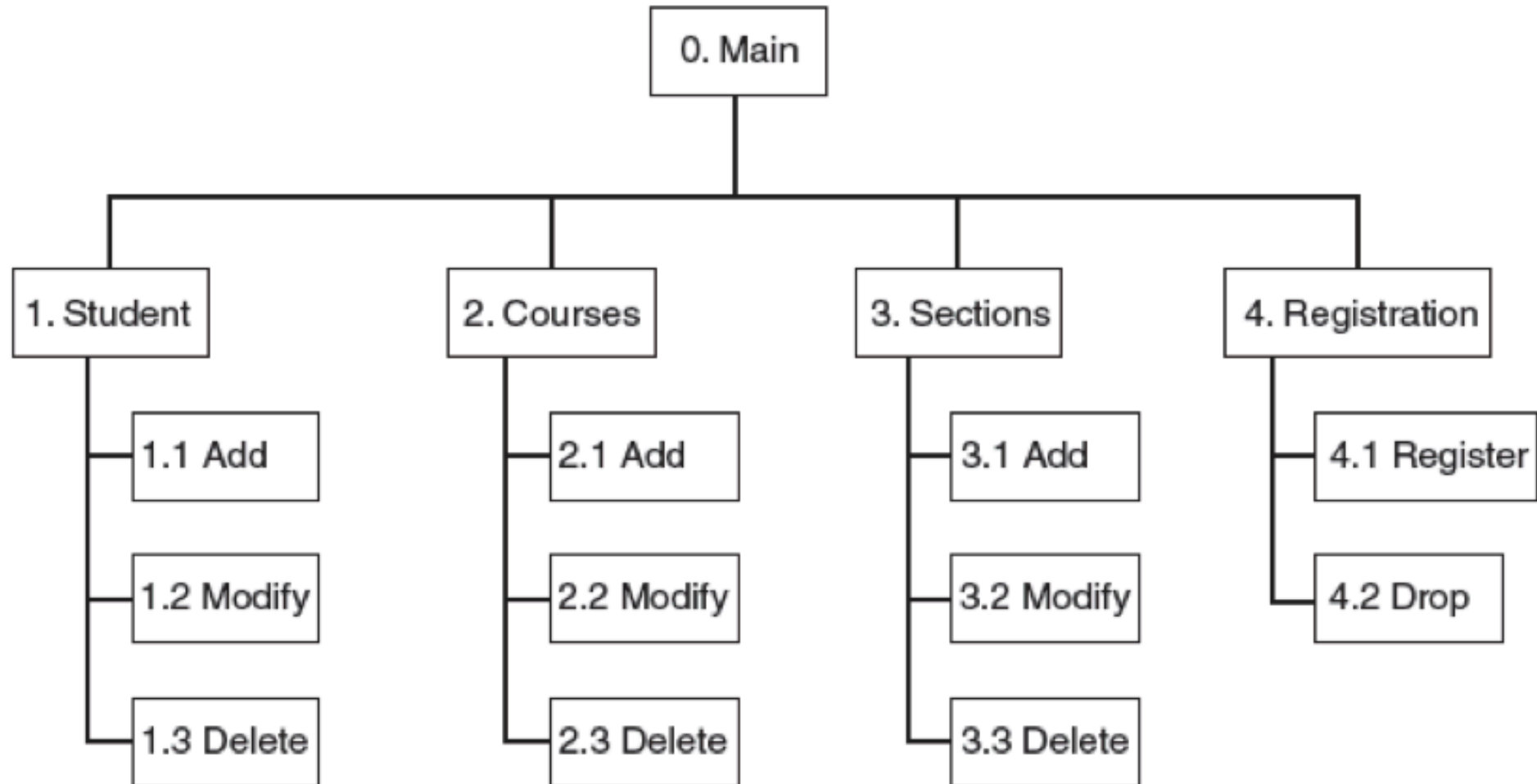


Functional Decomposition: Example - **Student Registration System**

- Design a system for managing course registration and Enrolments.
- Requirements specify four tasks
 - add, modify and delete students from the database
 - add, modify and delete courses from the database
 - add, modify, and delete sections for a given course
 - register and drop students from a section.
- Main module divided into four submodules
 - students, courses, sections, registration
- Decompose each into its tasks.



Functional Decomposition: Example- Student Registration System





- Many software systems must handle large amounts of data
- Data is stored in tables
 - row corresponds to an object or entity
 - columns correspond to attributes of the entities (basically an array of structs)
- Structured Query Language (SQL), a set of statements that
 - create the tables
 - add and modify data in the tables
 - retrieve data that match specified criteria

Relational Database Design



- Database design concentrates on
 - how to represent the data of the system, and
 - how to store it efficiently
- Data modeling
 - create a model showing the entities with their attributes, and how the entities are related to each other
- Logical database design
 - maps the model to a set of tables
 - relationships are represented via attributes called foreign keys
- Physical database design
 - deciding on types of attributes, how tables are stored, etc.



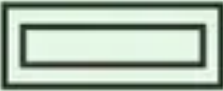
Relational Database Design



- Data modeling: ER diagram
 - Entities: rectangles
 - Attributes: ovals
 - Relationships: diamonds
- Identifier
 - attribute that has a unique value for each entity (underlined)
- Multi-valued attribute
 - can have several values at one time (double oval) i.e. email addresses

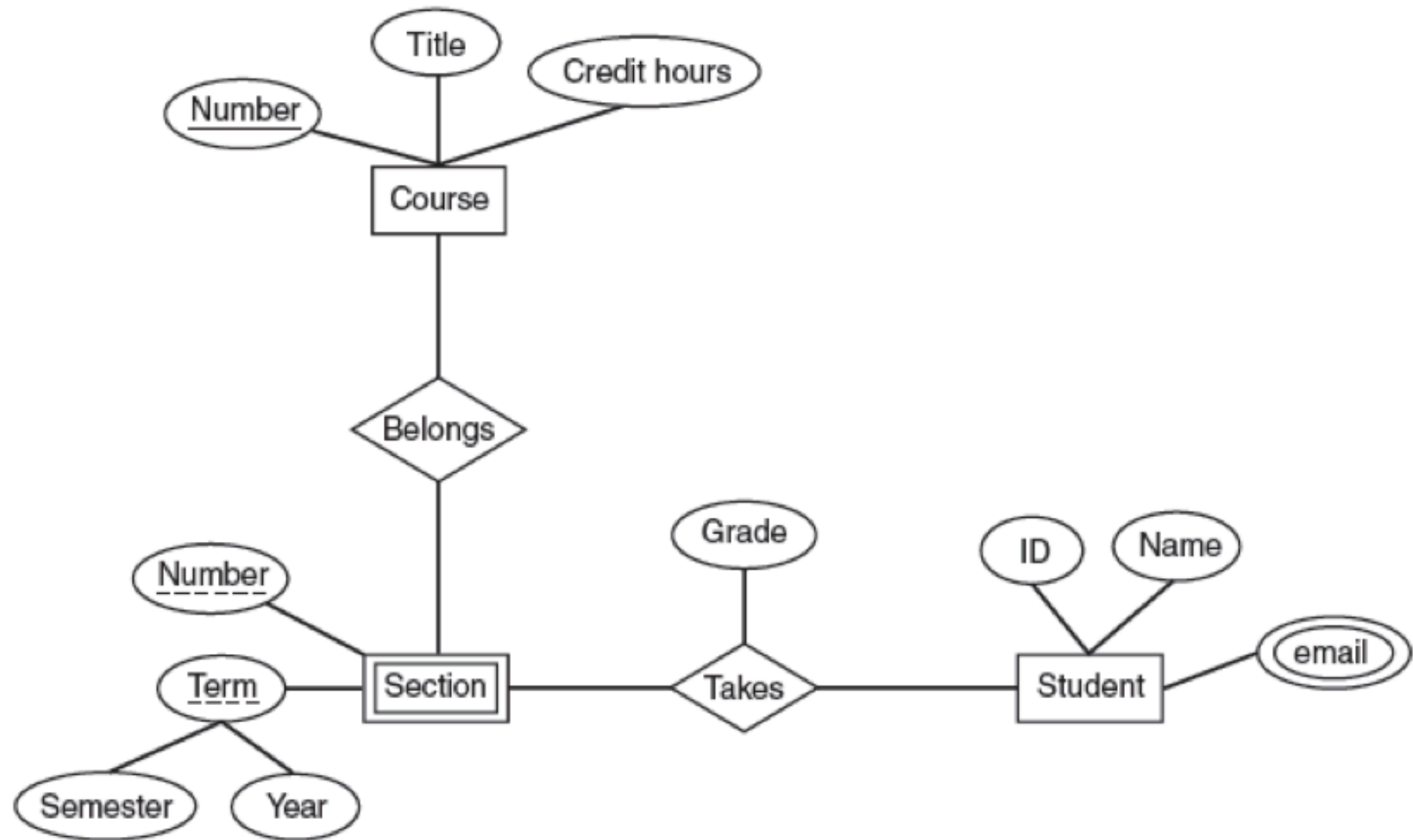


ER diagram components

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity



Relational Database design: ER diagram student registration system





Concept of Primary Key and Foreign Key -

Department

EmpNo	EmpName	DepNo
1001	Sahil	101
1004	Kavish	102
1006	Aditya	103
1005	Atul	104

← Foreign Key

Relationship

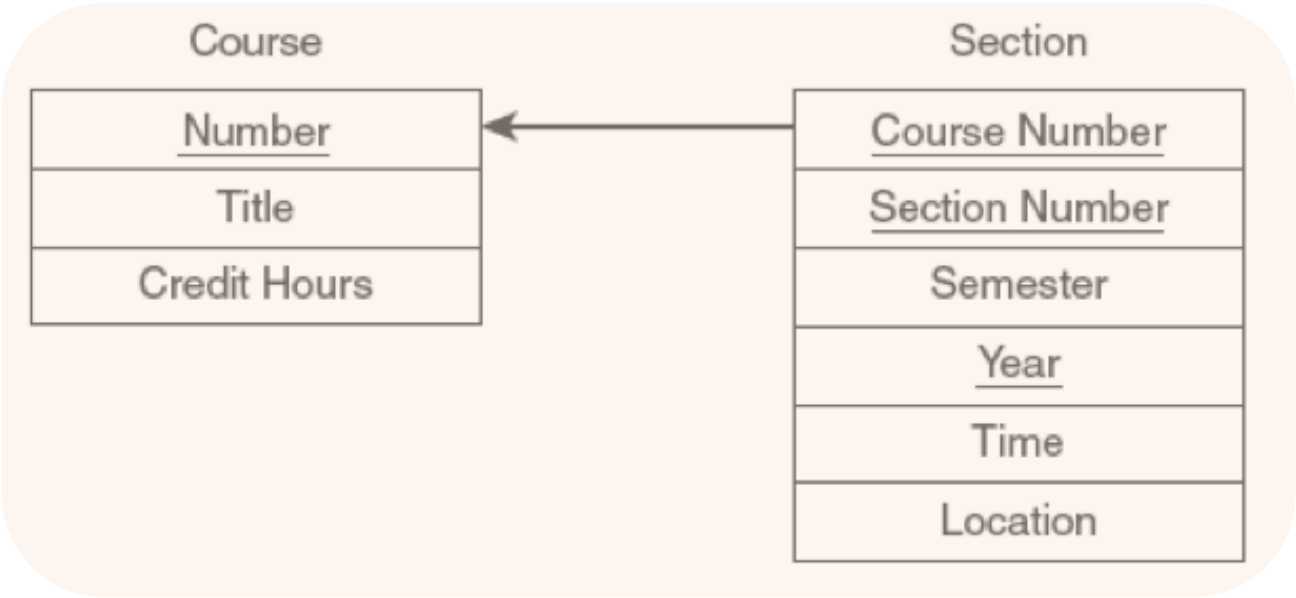
Employee

Primary Key →

DepNo	DName	Location
101	HR	Delhi
102	Sales	Bangalore
103	Marketing Executive	Hyderabad
104	Technical Engineer	Chennai

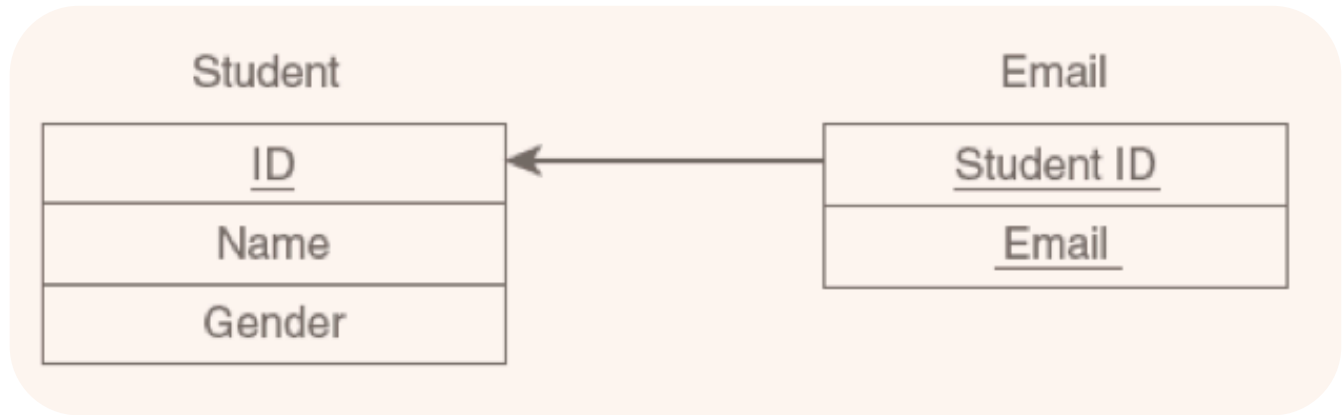
Student registration system: tables

A Relational Schema Diagram for Course and Section



Course Number is a foreign key, used to implement the “Belongs” relationship

A Relational Schema Diagram for Students and Email



Student ID is a foreign key, used to implement the multi-valued attribute



- Object-oriented system is made up of interacting objects
 - Maintain their own local state (private).
 - Provide operations over that state.
- Object-oriented design process involves
 - Designing classes (for objects) and their interactions.

Previous to the design phase:

- Requirements are usually expressed using use cases and use case diagrams.
- Preliminary class diagrams have often been produced during requirements analysis.

1. Requirements elicitation

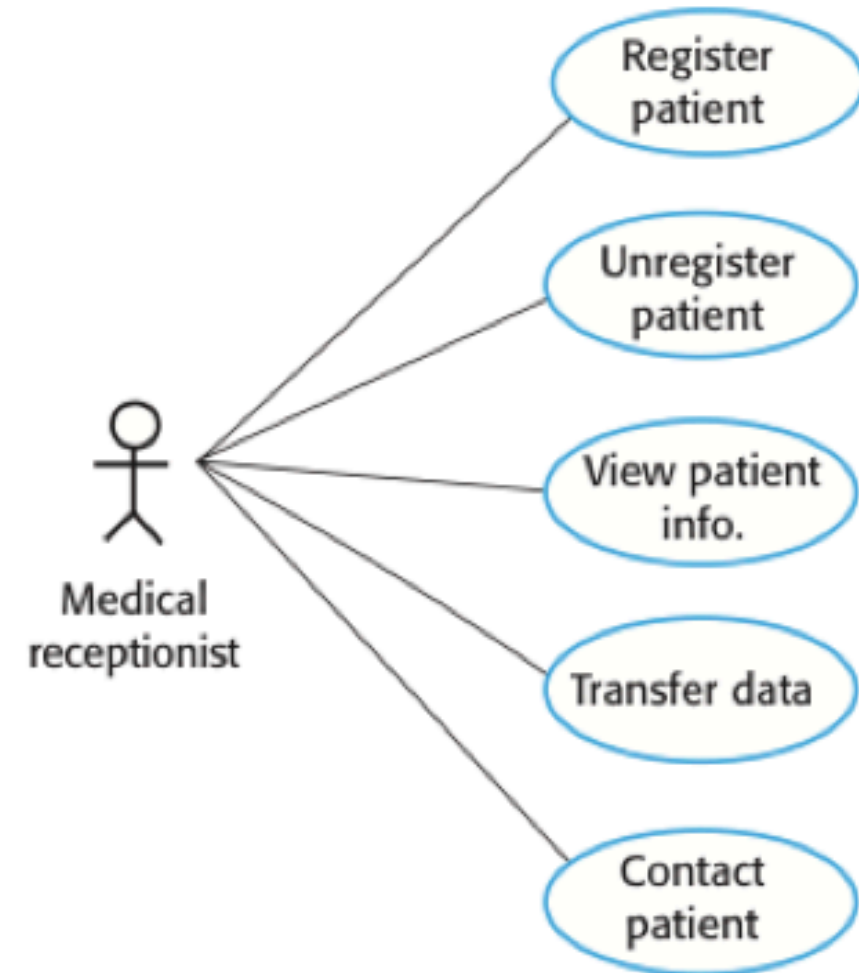


- Client and developers define the purpose of the system:

- Develop use cases
- Determine functional and non-functional requirements

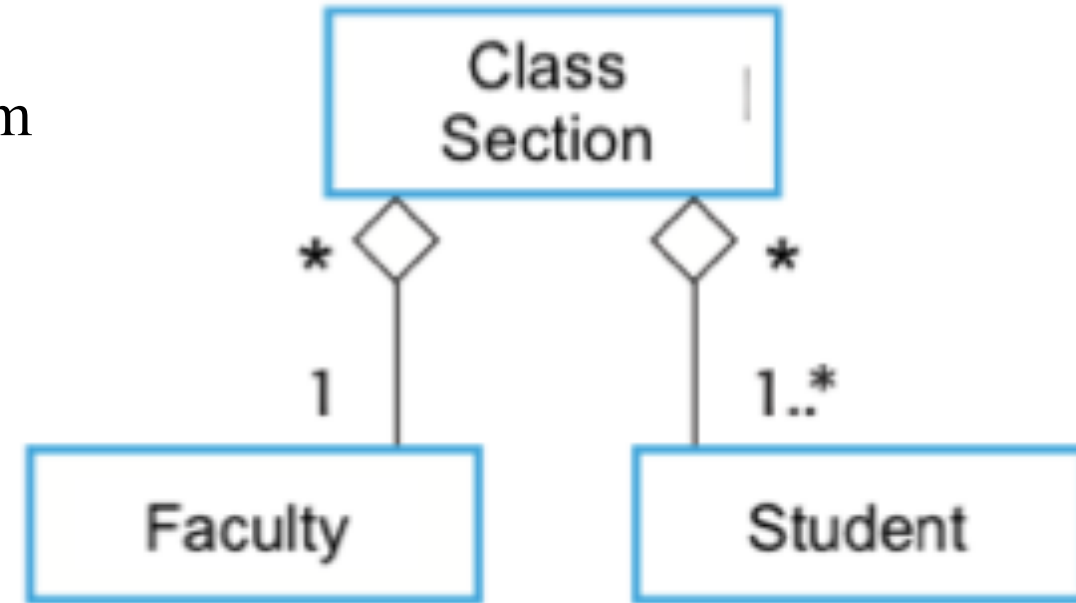
- Major activities

- Identifying actors.
- Identifying scenarios.
- Identifying use cases.
- Refining use cases.



2. Object Oriented Analysis

- Developers aim to produce a model of the system
 - Model is a class diagram
 - Describing real world objects (only)
- Goal: transform use cases to objects
- Major activities
 - Identifying objects: entities from the real world Look for nouns in use cases
 - Drawing the class diagram, with relationships Drawing state diagrams as necessary

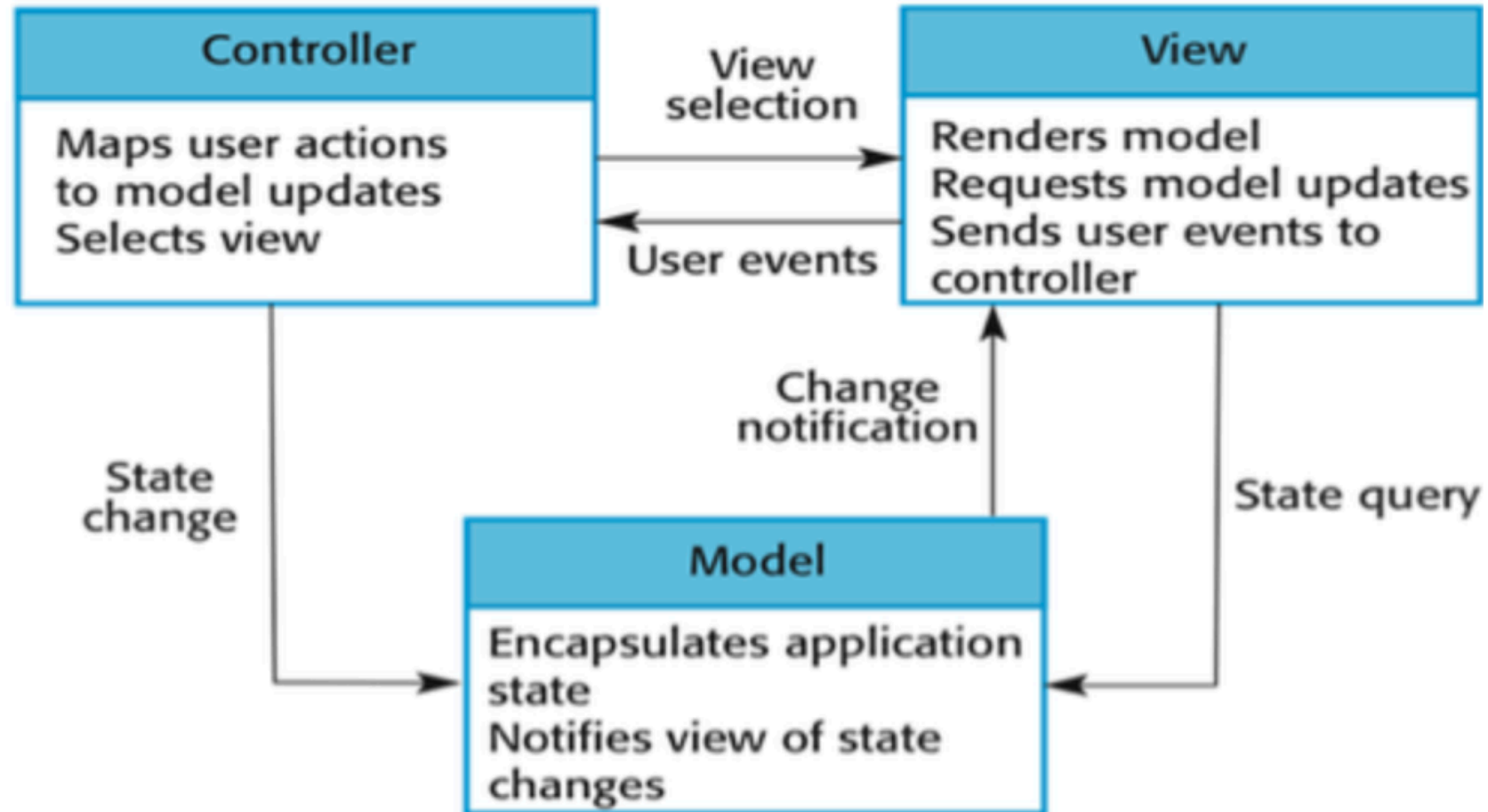


3. System Design (architecture)



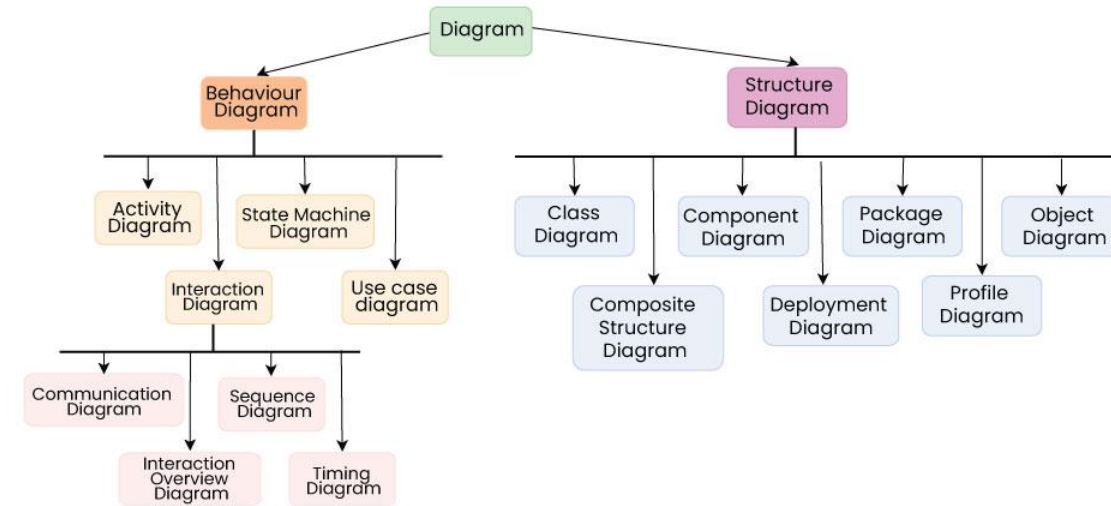
- Developers decompose the system into smaller subsystems
- Major activities
 - Identify major components of the system and their interactions (including interfaces).
 - Use architectural patterns
 - Identify design goals (non-functional requirements)
 - Refine the subsystem decomposition to address design goals

3. System Design (architecture)



4. Object Design

- Developers complete the object model by adding implementation classes to the class diagram.
- Major activities
 - **Interface specification:** define public interface of objects
 - **Reuse:** frameworks, existing libraries (code), design patterns (concepts)
 - **Restructuring:** maintainability, extensibility



5. Implementation

- Developers translate the class diagram into source code.
- Goal: map object model to code.
- Major activities
 - Map classes in model to classes in source language
 - Map associations in model to collections in source language
 - OO languages don't have “associations”
 - tricky: maintaining bidirectional associations Refactoring

```
#include <string>
#include <iomanip>
#include <sstream>
#include <iostream>
using namespace std;

// models a 12 hour clock
class Time //new data type
{
private:
    int hour;
    int minute;
    void addHour();

public:
    void setHour(int);
    void setMinute(int);
    int getHour() const;
    int getMinute() const;

    string display() const;
    void addMinute();
};

// class function implementations

void Time::setHour(int hr) {
    hour = hr; // hour is a member var
}
void Time::setMinute(int min) {
    minute = min; // minute is a member var
}
int Time::getHour() const {
    return hour;
}
int Time::getMinute() const {
    return minute;
}

void Time::addHour() { // a private member func
    if (hour == 12)
        hour = 1;
    else
        hour++;
}
```



- Some characteristics of a good software design:
 - **Consistency:**
 - ensure common terminology used across software elements.
 - common approach to help facility
 - common approach to error detection and diagnostic processing
 - **Completeness:**
 - All the requirements must be in the design
 - Design must include enough detail for the developers to know
 - what to do.



- Main goal: Simplicity
 - Easy to understand
 - Easy to change
 - Easy to reuse
 - Easy to test
 - Easy to code
- How do we measure simplicity of a design?
 - Coupling (goal: loose coupling)
 - Cohesion (goal: strong cohesion)

Before designing a database, it's crucial to understand important terms and concepts.

- A properly structured database guarantees efficiency, data accuracy, and usability.
- From understanding data storage to the principles that define data relationships, these concepts are essential for anyone involved in designing, managing, or optimizing databases.
- Whether you are new to database design or an experienced professional, these fundamental ideas serve as the building blocks for creating strong and scalable database systems.

What is database design?

- Database Design can be defined as a set of procedures or collection of tasks involving various steps taken to implement a database.
- A good database design is important.
- It helps you get the right information when you need it.
- Following are some critical points to keep in mind to achieve a good database design:
 - Data consistency and integrity must be maintained.
 - Low Redundancy
 - Faster searching through indices
 - Security measures should be taken by enforcing various integrity constraints.
 - Data should be stored in fragmented bits of information in the most atomic format possible.



Step 1: Determine the goal of your database and ensure clear communication with the stakeholders (if any). Understanding the purpose of a database will help in thinking of various use cases & where the problem may arise & how we can prevent it.

Step 2: List down all the entities that will be present in the database & what relationships exist among them.

Step 3: Organize the information into different tables such that no redundancy is there.

Step 4: Ensure uniqueness in every table. The uniqueness of records present in any relation is a very crucial part of database design that helps us avoid redundancy. Identify the key attributes to uniquely identify every row from columns. You can use various key constraints to ensure the uniqueness of your table, also keep in mind the uniquely identifying records must consume as little space as possible & shall not contain any NULL values.

Step 5: After all the tables are structured and information is organized, apply Normalization Forms to identify anomalies that may arise & redundancy that can cause inconsistency in the database.

Primary Terminologies Used in Database Design



- **Redundancy**: Redundancy refers to the duplicity of the data. There can be specific use cases when we need or don't need redundancy in our Database. For ex: If we have a banking system application then we may need to strictly prevent redundancy in our Database.
- **Schema**: Schema is a logical container that defines the structure & manages the organization of the data stored in it. It consists of rows and columns having data types for each column.
- **Records/Tuples**: A Record or a tuple is the same thing, basically its where our data is stored inside a table
- **Indexing**: Indexing is a data structure technique to promote efficient retrieval of the data stored in our database.
- **Data Integrity & Consistency**: Data integrity refers to the quality of the information stored in our database and consistency refers to the correctness of the data stored.
- **Data Models**: Data models provide visual modeling techniques to visualize the data & the relationship that exists among data. Ex: Network Model, Object Oriented Model, Hierarchical model.
- **Normalization**: The process of organizing data to reduce redundancy and dependency by dividing larger tables into smaller ones and defining relationships. It ensures data storage and consistency.

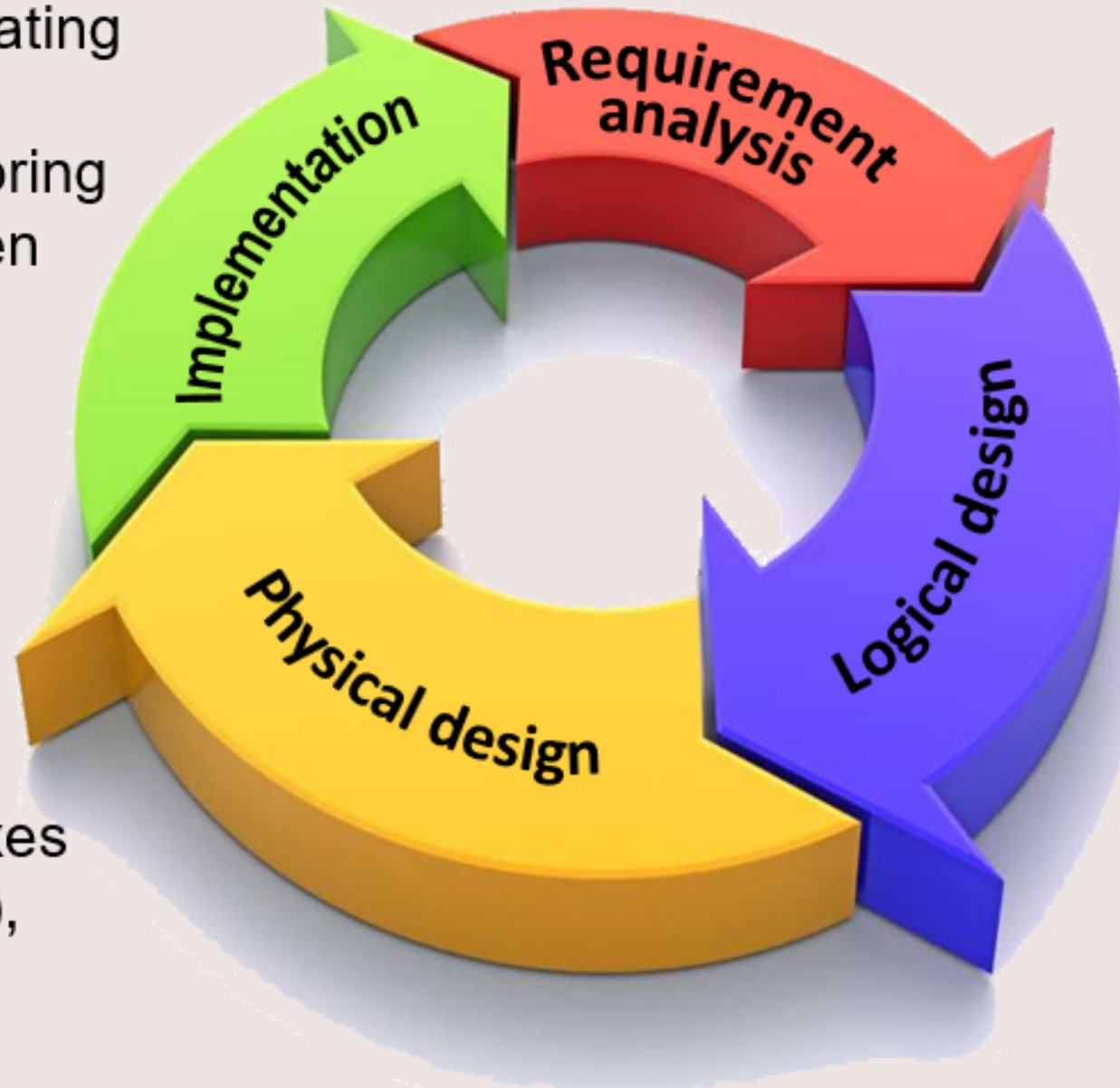


- **Functional Dependency:** Functional Dependency is a relationship between two attributes of the table that represents that the value of one attribute can be determined by another.
 - Ex: $\{A \rightarrow B\}$, A & B are two attributes and attribute A can uniquely determine the value of B.
- **Transaction:** Transaction is a single logical unit of work. It signifies that some changes are made in the database. A transaction must satisfy the ACID or BASE properties (depending on the type of Database).
- **Schedule:** Schedule defines the sequence of transactions in which they're executed by one or multiple users.
- **Concurrency:** Concurrency refers to allowing multiple transactions to operate simultaneously without interfering with one another.
- **Constraints:** Constraints are the rules applied to fields in a table to enforce data integrity. e.g., NOT NULL, UNIQUE, CHECK, etc. It ensures data quality and accuracy.

Database Design Lifecycle

- Creating and updating database
- Operation, monitoring
- Modifications when requirements change

- Selection of indexes (access methods), partitioning, and clustering of data



- Interviewing both the producers and users of data
- Data requirement specification
- Conceptual data modeling
- View integration
- Transformation of the conceptual data model into relational tables
- Normalization of tables

1. Requirement Analysis

It's very crucial to understand the requirements of our application so that you can think in productive terms. And imply appropriate integrity constraints to maintain the data integrity & consistency.

2. Logical & Physical Design

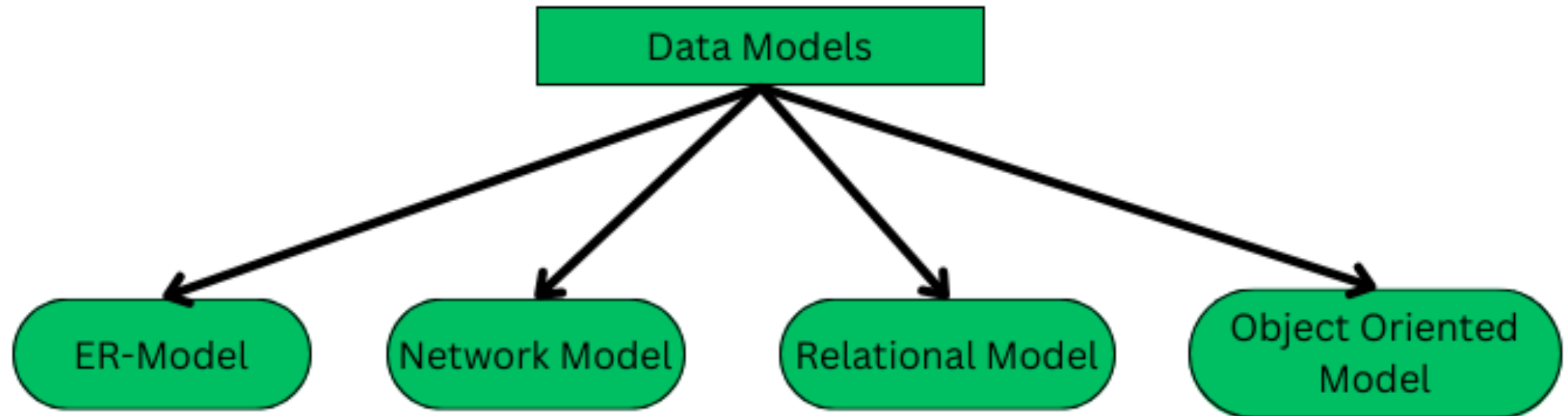
This is the actual design phase that involves various steps that are to be taken while designing a database. This phase is further divided into two stages:

- **Logical Data Model Design:** This phase consists of high-level design of our database based on initially gathered requirements to structure & organize our data accordingly. Entity, Key attributes identification & what constraints are to be implemented is the core functionality of this phase. It involves techniques such as Data Modeling to visualize data, normalization to prevent redundancy, etc.
- **Physical Design of Data Model:** This phase involves the implementation of the logical design made in the previous stage. All the relationships among data and integrity constraints are implemented to maintain consistency & generate the actual database.

Logical Data Model Design

The logical data model design defines the structure of data and what relationship exists among those data. The following are the major components of the logical design:

1. Data Models: Data modeling is a visual modeling technique used to get a high-level overview of our database. Data models help us understand the needs and requirements of our database by defining the design of our database through diagrammatic representation. Ex: ER-Model, Network model, Relational Model, object-oriented data model.



2. Entity: Entities are objects in the real world, which can have certain properties & these properties are referred to as attributes of that particular entity.

- There are 2 types of entities: Strong and weak entity, weak entity do not have a key attribute to identify them, their existence solely depends on one 1-specific strong entity & also have full participation in a relationship whereas strong entity does have a key attribute to uniquely identify them.

Weak entity example: **Loan** -> Loan will be given to a customer (which is optional) & the loan will be identified by the **Customer-id** -> to whom the lone is granted.

3. Relationships: How data is logically related to each other defines the relationship of that data with other entities. In simple words, the association of one entity with another is defined here. A relationship can be further categorized into - unary, binary, and ternary relationships.

- **Unary:** In this, the associating entity & the associated entity both are the same. **Ex:** Employee Manages themselves, and students are also given the post of monitor hence here the student themselves is a monitor.

- **Binary:** This is a very common relationship that you will come across while designing a database.

Ex: Student is enrolled in courses, Employee is managed by different managers, One student can be taught by different professors.

- **Ternary:** In this, we have 3 entities involved in a single relationship.

Ex: an employee works on a project for a client. Note that, here we have 3 entities: Employee, Project & Client.



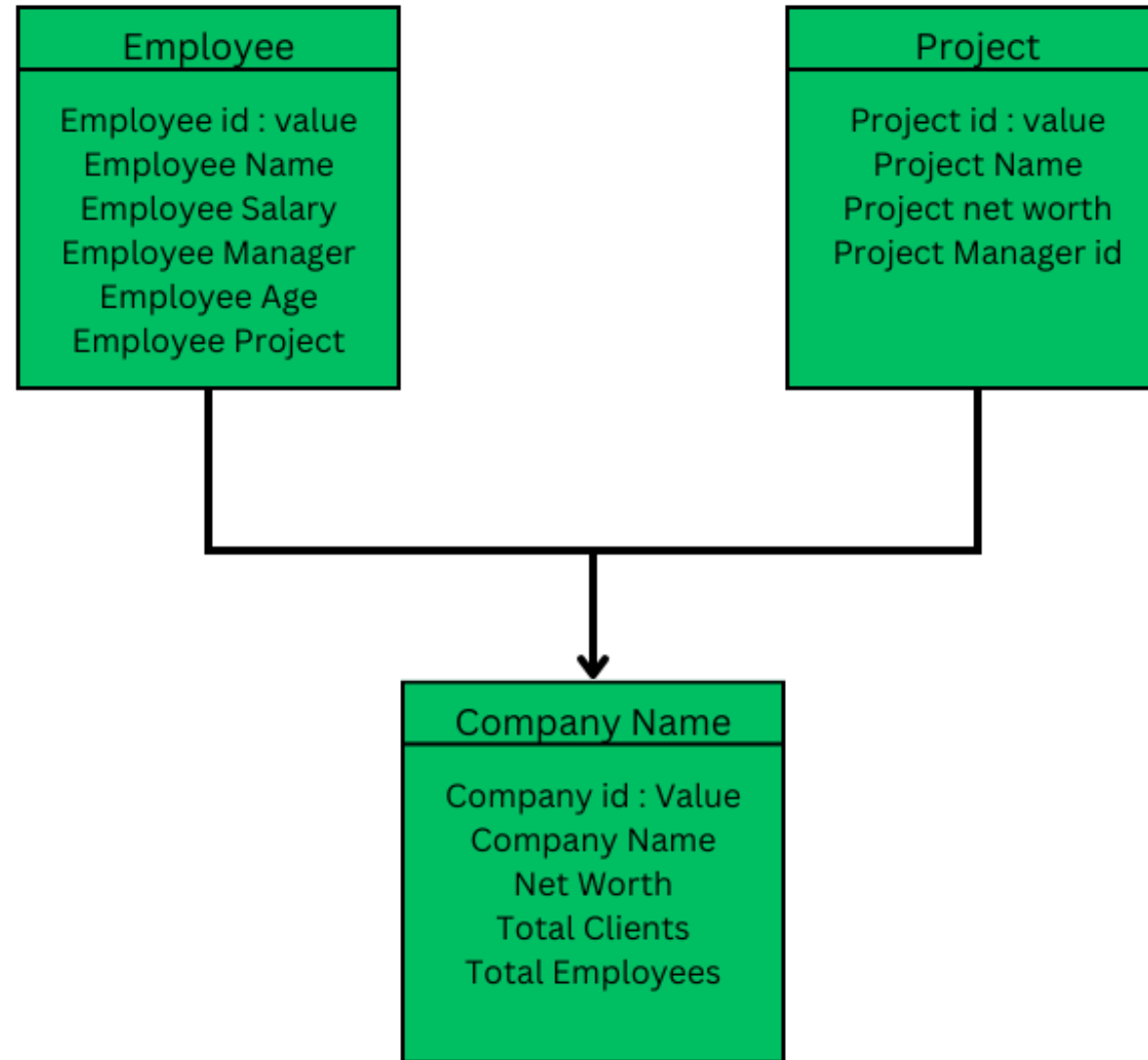
4. Attributes: Attributes are nothing but properties of a specific entity that define its behavior.

For example, an employee can have unique_id, name, age, date of birth (DOB), salary, department, Manager, project id, etc.

5. Normalization: After all the entities are put in place and the relationship among data is defined, we need to look for loopholes or possible ambiguities that may arise due to **CRUD** operations.

- To prevent various Anomalies such as INSERTION, UPDATION, and DELETION Anomalies.
- Data Normalization is a basic procedure defined for databases to eliminate such anomalies & prevent redundancy.

Logical Design Example



Logical Design

- The main purpose of the physical design is to implement the logical design that is to show the structure of the database along with all the columns & their data types, rows, relations, relationships among data & clearly define how relations are related to each other.
- Following are the steps taken in physical design-

Step 1: Entities are converted into tables or relations that consist of their properties (attributes)

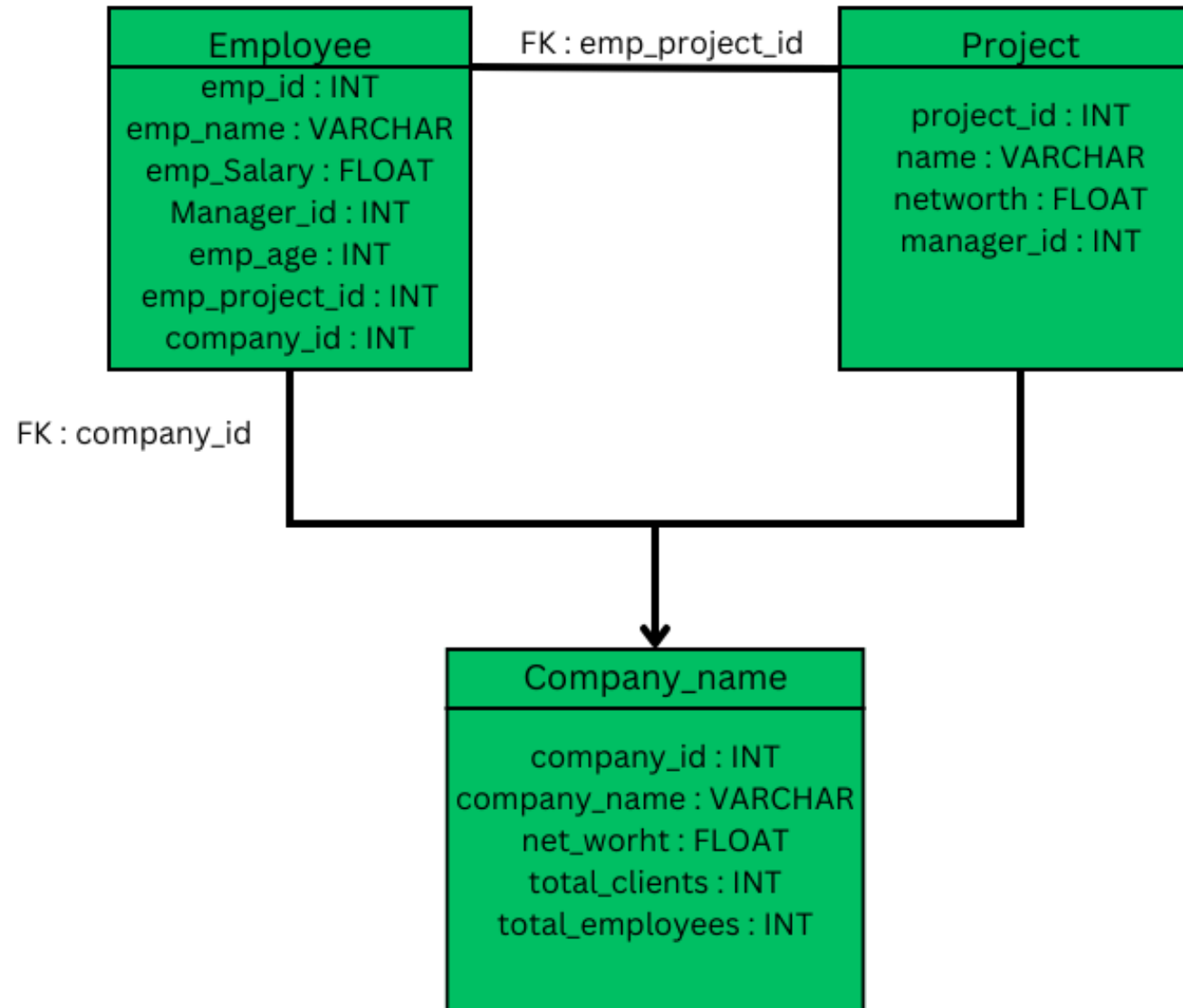
Step 2: Apply integrity constraints: establish foreign key, unique key, and composite key relationships among the data. And apply various constraints.

Step 3: Entity names are converted into table names, property names are translated into attribute names, and so on.

Step 4: Apply normalization & modify as per the requirements.

Step 5: Final Schemes are defined based on the entities & attributes derived in logical design.

Physical Design Example



Physical Design

- ❖ A good database design is an essential part of a strong database management system (DBMS).
- ❖ It provides the basis for data governance, data storage, and data retrieval.
- ❖ The quality of a database has a direct impact on a system's overall performance and dependability.
- ❖ It is important to consider data organization, standardization, performance, integrity, and more when designing a database to meet the needs of your organization and your users.

- **Data Integrity:** Ensures data accuracy and consistency by preventing redundancy and enforcing constraints.
- **Efficiency:** Enables fast data access and retrieval through optimized queries.
- **Scalability:** Allows for easy expansion of the database to accommodate future data growth.
- **Maintainability:** Simplifies database updates and modifications.

